MISIÓN 02 · INVESTIGACIÓN DOCUMENTAL · 5 DE NOVIEMBRE DE 2026

Diseño de la arquitectura fundamental de Nexo

Una infraestructura tecnológica propia — identidad, memoria, principios, autoridad y continuidad — independiente de modelos de IA, proveedores, dispositivos, sistemas operativos y conectividad.

Este documento no diseña un chatbot avanzado ni un agente que usa herramientas. Profundiza donde la Misión 01 dejó preguntas abiertas: cómo se construye el sustrato que sobrevive al cambio — y a la desaparición — de todo lo que hoy lo rodea. La Misión 01 queda como antecedente y referencia; ninguna tecnología, framework o decisión propuesta en ella queda aprobada por este documento.

`Alcance: arquitectura conceptual, sin implementación`

`No modifica NCS M0 ni P07`

`Fuentes consultadas el 2026-11-05 · lectura completa e índice distinguidos en las referencias`

`Ninguna decisión tomada en nombre del propietario`

Reglas de lectura. Cada afirmación sustantiva lleva una etiqueta:

EVIDENCIA documentada en norma, especificación o despliegue comprobable · PROPUESTA diseño plausible con piezas existentes, sin aval como conjunto · HIPÓTESIS conjetura razonable sin prueba en este contexto · UNKNOWN no se encontró documentación primaria que lo resuelva.

Una capacidad teórica nunca se presenta como capacidad existente. Donde la investigación no pudo verificar una fuente primaria, se dice: varias referencias se obtuvieron vía índice de búsqueda (`index`) y no por lectura completa (`verified live`).

Tesis general PROPUESTA síntesis derivada de la evidencia de las secciones siguientes

La independencia *total* no existe como producto; existe como **arquitectura de dependencias explícitas y sustituibles**. Cada capa —identidad, memoria, cognición, supervivencia, evolución, aislamiento— tiene piezas estándar maduras, pero ninguna resuelve sola la continuidad, y varias garantías deseadas son contradictorias entre sí o imposibles sin recursos externos. El diseño honesto es un inventario de dependencias con coste de sustitución conocido, no una promesa de autosuficiencia.

§ Resumen ejecutivo

Qué encontró esta investigación, en diez líneas verificables

Qué encontró esta investigación, en diez líneas verificables

1. EVIDENCIA **La identidad puede desacoplarse del proveedor hoy** — con identificadores descentralizados (DID, Recomendación W3C desde 2022) y credenciales verificables (VC 2.0, 2025) — pero ningún método ofrece a la vez operación offline, rotación de claves y ausencia de terceros de confianza.
2. EVIDENCIA **La memoria duradera es un problema resuelto en piezas**: formatos abiertos (texto UTF-8/JSONL, SQLite con compatibilidad comprometida hasta 2050), registros append-only con hash encadenado y árboles de Merkle (patrón Certificate Transparency, RFC 9162), y sincronización P2P sin nube (CRDT, Syncthing, git). No existe, en cambio, un estándar de «memoria de agente».
3. EVIDENCIA **La cognición es intercambiable en el sobre, no en la semántica.** La API compatible con OpenAI y el protocolo MCP permiten cambiar de proveedor o de herramienta; los formatos ONNX y GGUF permiten mover pesos. Pero el comportamiento, la calibración y la interpretación de la Constitución no se transfieren con garantías medibles.
4. EVIDENCIA **La evaluación pública no decide.** El mismo modelo puntúa distinto según el harness (hasta 5 puntos en MMLU), el formato del prompt mueve la exactitud 2–5 puntos y los benchmarks se contaminan en 6–12 meses. Cambiar de modelo sin evaluación propia es una hipótesis, no una mejora.
5. EVIDENCIA **La supervivencia es disciplina, no tecnología exótica**: regla 3-2-1-1-0, copias inmutables, RTO/RPO escritos, restauraciones de prueba periódicas y un kit de arranque que

no dependa del sistema perdido.

6. EVIDENCIA **La independencia física total es imposible sin recursos externos.** Fabricar un chip avanzado exige más de 1.000 pasos, litografía EUV y una cadena que ningún país domina completa. Lo alcanzable es degradación elegante: seguir funcionando, peor, con lo que hay.
7. PROPUESTA **El Core debe ser mínimo y sin dependencia de modelo**: identidad, Constitución versionada y firmada, registro append-only y memoria canónica. Todo lo demás —modelos, herramientas, transporte— vive detrás de adaptadores desechables con contratos versionados.
8. PROPUESTA **Labs se aísla por compartimentación real** (VM, no solo contenedores), recibe capacidades atenuadas con caducidad corta —nunca claves del Core— y todo su resultado entra al núcleo como propuesta a verificar, nunca como hecho.
9. UNKNOWN **No existe estándar de Constitución de agente** ni de versionado de autoridad, ni un test que verifique que un agente «sigue siendo el mismo» tras migrar memoria, modelo y formato. La continuidad es hoy procedencia criptográfica más evaluación de comportamiento.
10. EVIDENCIA **Siete contradicciones estructurales no se resuelven: se arbitran** (ver sección propia). Todo diseño que prometa eliminarlas está vendiendo una de las dos mitades.

---

01 Nexo Core

Identidad, Constitución, autoridad y memoria fundamental que

sobreviven a la desaparición de todos los modelos externos

1.1 Identidad criptográfica: el ancla que no es un proveedor

EVIDENCIA Un **DID** es una URI `did:<método>:<id>` que resuelve a un documento con material criptográfico, métodos de verificación y servicios. El controlador prueba el control sin permiso de un registro central, IdP o CA. W3C DID Core v1.0 es Recomendación desde el 19-07-2022 y no presupone tecnología ni criptografía concreta: el método define creación, lectura, actualización y desactivación. w3.org/TR/did-core

- **Ventajas:** desacoplamiento del proveedor de identidad; portabilidad entre sistemas que soporten el método; múltiples DIDs por entidad para separar contextos; prueba por firma en lugar de contraseña de plataforma.
- **Limitaciones:** un DID no prueba identidad legal, veracidad de una afirmación ni fiabilidad de un endpoint —eso lo decide el verificador con su política—. La interoperabilidad real depende del método concreto: en la publicación había 103 especificaciones de método experimentales, 32 drivers y 46 implementaciones. Fragmentación real.
- **Riesgo:** elegir un método inmaduro o de un solo implementador es cambiar un vendor por otro con otro nombre.

1.2 `did:key`: identidad = clave pública, con un límite duro

EVIDENCIA Método no registral y puramente generativo (W3C

CCG, v0.9, no Recomendación W3C): el DID es la clave pública codificada. Resolución determinista, inmediata, sin red, a coste cero de infraestructura, totalmente offline. w3c-ccg.github.io/did-key-spec

> **Límite documentado en la propia especificación**
>
> Como el DID no está en ningún registro, **no puede actualizarse ni desactivarse**. Rotar la clave implica un DID nuevo: una identidad nueva. La especificación lo recomienda solo para usos de días/semanas, o de larga duración únicamente con alta confianza en aislamiento por hardware. Conclusión para Nexo: `did:key` es un **ancla de arranque**, no una identidad longeva. La pérdida o compromiso de la clave privada es pérdida o usurpación total, sin vía de recuperación dentro del método.

1.3 Métodos con registro: rotación a cambio de dependencia

EVIDENCIA PROPUESTA Los métodos registrales permiten publicar un documento DID nuevo —nueva clave— bajo el mismo DID, y desactivarlo. `did:web` lo ancla a un dominio HTTPS (`/.well-known/did.json`); los métodos ledger, a una cadena; otros, a registros P2P. Pero `did:web` reintroduce dependencia de DNS/TLS/registrador, y un ledger reintroduce la dependencia de esa red, su gobernanza y su coste.

UNKNOWN No hay método gratuito en los tres ejes a la vez —offline, rotatable y sin tercero de confianza—. Esa tríada como propiedad

simultánea demostrada es **UNKNOWN**: es una tensión de diseño, no un fallo de implementación. **Portabilidad ≠ estabilidad.**

1.4 Autoridad portable: credenciales verificables y capacidades

EVIDENCIA **Credenciales verificables (VC 2.0)**, Recomendación W3C desde mayo de 2025: credencial a prueba de manipulación con triángulo emisor–titular–verificador. Una credencial emitida por A puede presentarse a B sin que A esté en línea. Límites: la verificación criptográfica no establece que la afirmación sea verdadera; los perfiles de codificación son incompatibles entre sí (un verificador VC-JWT 1.1 no verifica una VC 2.0 JOSE/COSE), y sin mecanismo de estado/revocación, una credencial es válida hasta su caducidad aunque la autoridad haya cambiado de opinión. w3.org/TR/vc-data-model-2.0

EVIDENCIA PROPUESTA **Capacidades delegables** (UCAN, trabajo comunitario v1.0, sin estatus IETF/W3C; ZCAP-LD, borrador CCG v0.4.0): autoridad acotada y firmada —recurso, acción, caducidad, cadena de pruebas— que el titular puede *atenuar* pero nunca ampliar, y que funciona sin servidor de autorización en línea. Límites: sin estándar único, las cadenas no interoperan entre implementaciones y la revocación offline es débil. Y una distinción que debe vivir en la Constitución, no solo en el código: **probar quién firma no prueba qué puede hacer** —autenticación ≠ autorización—.

1.5 Custodia de claves: el punto que decide todo

EVIDENCIA NIST SP 800-63B-4 (julio 2025) gradúa la

EVIDENCIA NIST SP 800-63B-4 (julio 2025) gradúa la autenticación en AAL1/AAL2/AAL3. AAL3 exige autenticador hardware con clave privada **no exportable** y resistencia a phishing; los autenticadores sincronizables **no** se permiten en AAL3. De ahí una contradicción documentada, no resoluble por diseño, solo por elección:

> **Sincronizable y no-exportable son mutuamente excluyentes para la misma clave**
>
> O la identidad raíz sobrevive al cambio de dispositivo porque la clave puede copiarse —y entonces puede ser exfiltrada—, o sobrevive porque está en hardware que no la deja salir —y entonces no sobrevive a la pérdida de ese hardware sin un mecanismo externo—. No hay tercera opción criptográfica; hay arquitecturas que reparten el riesgo:

- EVIDENCIA **Shamir / SLIP-39:** dividir el secreto en *n* partes con umbral *k*; menos de *k* revelan cero información (Shamir, 1979). BIP-39 no soporta Shamir nativamente; SLIP-39 usa 20/33 palabras por parte y es incompatible con BIP-39. Advertencia: implementaciones no estándar de Shamir-sobre-BIP39 han mostrado colapso de umbral (un 3-de-5 recuperable con 2 partes en una implementación concreta, issue público de 2025). La teoría es sólida; **la implementación es el riesgo**.
- PROPUESTA **Multifirma en el documento DID:** varias claves de control con umbral; rotar una sin cambiar el DID, solo en métodos registrales. Requiere gobernanza explícita de quién firma la rotación.

- EVIDENCIA **Copia cifrada offline + frase** (práctica 3-2-1-1-0, §08): la frase es un secreto único —quien la tiene, es la identidad—.

HIPÓTESIS La recuperación de identidad es un problema **humano y físico** —dónde están las partes, quién las custodia, qué pasa si el custodio muere o se incapacita—, no un problema que una especificación resuelva. Cualquier diseño que lo ignore es hipótesis, no ingeniería.

1.6 Constitución versionada: piezas reales, estándar inexistente

UNKNOWN **Hallazgo negativo importante:** no se encontró ninguna norma W3C, IETF o ISO que defina el versionado de una «Constitución» de agente, su ratificación, su derogación o su enlace criptográfico a una identidad. Lo que existe son piezas reutilizables:

- EVIDENCIA **Git + firmas:** historial append-only con hash encadenado por commit y firmas GPG/SSH/Sigstore. Versiona *texto*; no versiona *autoridad*: quién puede firmar una nueva versión debe estar definido en la Constitución anterior —la cláusula de enmienda— y ser verificable por un tercero sin confiar en el autor del cambio.
- EVIDENCIA **Procedencia SLSA / in-toto:** atestaciones autenticadas sobre artefactos (builder, fuente, definición de build, digest). SLSA v1.0 (2023), v1.1 y v1.2 (2025). Límite documentado: prueba *de dónde vino* un artefacto y *cómo se construyó*, no si el código es seguro ni si la Constitución es

correcta.

Diseño propuesto para el Core PROPUESTA

Constitución como documento versionado (semver + hash), cada versión firmada por la clave raíz vigente, con cláusula de enmienda explícita —quórum, claves autorizadas, vigencia— y cada decisión referenciando `constitución_id + versión + hash`. El núcleo verifica la Constitución *antes* de actuar: un cambio de modelo, dispositivo o proveedor no la altera, y un cambio de Constitución sin firma válida es rechazado aunque lo pida el modelo. Es una **política de verificación del núcleo**, no una propiedad del modelo.

HIPÓTESIS La Constitución como texto sobrevive a cualquier cambio. Su *interpretación* —cómo un modelo la aplica en casos ambiguos— no es portable ni está estandarizada, y no existe métrica de «fidelidad constitucional» entre modelos.

Tabla 1 — Piezas de identidad del Core: estado, ventaja y límite duro

TECNOLOGÍA	ESTADO
DID Core v1.0	

ESTADO

Evidencia

VENTAJA

Identidad sin registro central

LÍMITE

No prueba verdad ni confianza; depende del método

TECNOLOGÍA

did:key

ESTADO

Evidencia

VENTAJA

Offline, determinista, sin infraestructura

LÍMITE

Sin rotación ni revocación; clave = identidad

TECNOLOGÍA

Método registral (web / ledger)

ESTADO

Evidencia

VENTAJA

Rotación y revocación, mismo DID

LÍMITE

Reintroduce DNS, ledger o gobernanza

TECNOLOGÍA

VC 2.0

ESTADO

Evidencia

VENTAJA

Autoridad portable y verificable

LÍMITE

Perfiles incompatibles; la revocación necesita estado

TECNOLOGÍA

UCAN / ZCAP-LD

ESTADO

Evidencia desigual

VENTAJA

Delegación atenuable offline

LÍMITE

Sin estándar único; revocación offline débil

TECNOLOGÍA

Shamir / SLIP-39

ESTADO

Evidencia

VENTAJA

Recuperación con umbral, sin punto único

LÍMITE

Implementación frágil; custodia humana

TECNOLOGÍA

Versionado de Constitución

ESTADO

Unknown como estándar

VENTAJA

Texto + firma + cláusula de enmienda (piezas)

LÍMITE

La interpretación por el modelo no es portable

02

Nexo Cognition

Razonamiento intercambiable: modelos locales, externos o futuros como piezas sustituibles, nunca como identidad

2.1 La interfaz neutral: un dialecto interno, adaptadores desechables

EVIDENCIA La **API compatible con OpenAI** (`/v1/chat/completions`) es el estándar *de facto*: mismo endpoint, `messages[]` , streaming. En el caso simple, cambiar de proveedor es cambiar `base_url` + `api_key` + `model` . Pero es compatibilidad de *sobre*, no de *funcionalidad*: structured outputs, tool calling, visión, contexto y streaming difieren por proveedor. No es un estándar de un organismo; es una convención que un proveedor puede romper, y no hay garantía de que el modelo tras un nombre sea el mismo mañana.

EVIDENCIA **MCP** (Model Context Protocol), protocolo abierto JSON-RPC 2.0 para herramientas, recursos y prompts: publicado el 25-11-2024 y donado el 09-12-2025 a la Agentic AI Foundation (Linux Foundation). Su revisión vigente, 2026-07-28, es sin estado, elimina el handshake `initialize` y depreca Roots/Sampling/Logging y HTTP+SSE —es decir, **rompe compatibilidad con revisiones anteriores** en puntos concretos—. Fijar la revisión es obligatorio. MCP tampoco estandariza la *semántica* de cada herramienta ni la calidad del modelo al elegirla. modelcontextprotocol.io/specification/2026-07-28

UNKNOWN El estado normativo de **A2A / Agent2Agent** no se verificó en fuente primaria en esta investigación; este documento no afirma nada sobre él.

> **Consecuencia de diseño** PROPUESTA
>
> El núcleo habla **un dialecto interno estable** —mensajes, herramientas con JSON Schema 2020-12, resultados y errores tipados— y cada proveedor o revisión entra por un **adaptador**. El adaptador es desechable; el dialecto interno es la Constitución técnica. Si el dialecto interno es la API de un proveedor, el núcleo *es* ese proveedor.

2.2 Intercambio de pesos: ONNX, GGUF, safetensors

- EVIDENCIA **ONNX** (LF AI, desde 2017): formato abierto de intercambio —grafo de operadores, tensores, pesos, metadatos — con versionado independiente de IR, opset y modelo. Permite entrenar en un framework y ejecutar en otro runtime/hardware. Límites: no todo modelo u operador es representable; la deriva de opset entre exportador y runtime rompe modelos; un fichero válido puede fallar si el runtime no tiene el kernel o cambia la precisión. Y para LLM de frontera, ONNX no es el formato habitual de despliegue.
- EVIDENCIA **GGUF** (desde 2023, formato de facto de llama.cpp/ggml, sin organismo de estándares): un solo fichero con arquitectura, contexto, tokenizador y plantilla de chat; tensores cuantizados; despliegue local en CPU, Apple Silicon y

tensores cuantizados, despliegue local en CPU, Apple Silicon y GPU de consumo sin nube. Límites: atado a los tipos de ggml; el vocabulario embebido puede ser menos preciso que el tokenizador original; la cuantización cambia el comportamiento.

- EVIDENCIA **safetensors**: formato de pesos seguro —sin ejecución de código al cargar, a diferencia de pickle—. No se verificó en mayor profundidad en esta investigación.

EVIDENCIA **Contradicción:** el formato portable (ONNX) no cubre bien los LLM actuales; el formato que sí los cubre localmente (GGUF) no es un estándar formal. Y el acceso a los pesos de frontera es decisión del proveedor, no una propiedad técnica: la intercambiabilidad real de pesos de frontera es hoy limitada.

2.3 Evaluación independiente: la única defensa contra el cambio silencioso

EVIDENCIA Existen harnesses abiertos —HELM (Stanford, 16 escenarios, 7 métricas) y lm-evaluation-harness (60+ benchmarks) —, pero la evidencia sobre sus límites es contundente:

- El mismo modelo puntúa distinto según el harness: Claude 2.1 perdió 5 puntos en MMLU y GPT-4, 4 puntos, entre HELM y la evaluación interna (Stanford CRFM, 2024).
- El formato solo (`A.` vs `(A)` vs `A)`) mueve la exactitud 2–5 puntos; la sensibilidad al prompt puede moverla ≥10 puntos.
- Contaminación: si el test estaba en el entrenamiento, la puntuación mide memorización. Los benchmarks tienen vida útil de 6–12 meses.

- Saturación y ruido: MMLU supera el 90% en frontera desde 2024, con 5%+ de errores de etiqueta en algunos subconjuntos; GPQA Diamond (198 preguntas) tiene un intervalo de confianza del 95% de ±6,4 puntos —una ventaja de 3 puntos no es un ranking—.

Regla operativa PROPUESTA

La evaluación pública sirve para **descartar** —una lista corta de 2–3 candidatos—, no para **decidir**. Decidir exige evaluación propia: 200–500 casos de la tarea real, mismo harness, mismas plantillas, re-ejecutada en cada cambio de snapshot, con metadatos de validez (fecha, harness, prompt, semilla, versión del modelo). Sin eso, «cambiamos de modelo y es mejor» es hipótesis. Además, el núcleo debe registrar `modelo_id + snapshot/fecha + digest si disponible` en cada decisión: el cambio silencioso de snapshot por el proveedor es indistinguible de un cambio de modelo si no se registra.

2.4 Qué se pierde al cambiar de modelo

Tabla 2 — Transferencia entre modelos, por capas

CAPA	
Texto: Constitución, memoria, herramientas, prompts	

¿SE TRANSFIERE?

Sí, si están en formatos abiertos versionados

ESTADO

Evidencia (si se diseñó así)

CAPA

Comportamiento: esquemas JSON, tool calling, multi-turno, uso de contexto

¿SE TRANSFIERE?

Parcialmente; depende del modelo y del prompt

ESTADO

Evidencia de variabilidad

CAPA

Calibración: cuándo pedir aclaración, cuándo no actuar, prudencia

¿SE TRANSFIERE?

Sin garantía de portabilidad

ESTADO

Hipótesis

CAPA

Conocimiento latente en los pesos

¿SE TRANSFIERE?

No; es el modelo

ESTADO

Evidencia

CAPA

Interpretación de la Constitución en casos ambiguos

¿SE TRANSFIERE?

No estandarizada

ESTADO

Unknown

CAPA

Historial de decisiones y su justificación

¿SE TRANSFIERE?

Sí, si vive en el registro append-only (§03)

ESTADO

Propuesta

HIPÓTESIS No existe una métrica estándar de «pérdida por cambio de modelo». La tabla anterior es una hipótesis estructurada sobre dónde mirar, no una medición.

03

Memoria e identidad

Continuidad real entre dispositivos, sistemas y generaciones de modelos: procedencia, integridad, recuperación y protección contra corrupción o manipulación

3.1 Formatos que se podrán leer en 2050

EVIDENCIA Principio OAIS (ISO 14721:2012): la preservación no es el medio, es la **interpretabilidad futura**. OAIS es un marco conceptual —Ingest, Almacenamiento, Gestión, Planificación de preservación, Acceso, Administración; paquetes SIP/AIP/DIP—, no una implementación. La estrategia es migración controlada o emulación: la migración puede perder semántica; la emulación exige mantener emuladores indefinidamente

exige mantener enlaces indefinidamente.

- EVIDENCIA **Texto plano UTF-8 + JSON/JSONL:** legible sin herramienta propietaria, buscable, comparable en git. Es la capa de máxima durabilidad. Límite: sin esquema versionado, JSON es solo sintaxis; la semántica vive en documentación externa.
- EVIDENCIA **SQLite:** un solo fichero, portable entre arquitecturas, transacciones atómicas, formato documentado y retrocompatible hasta 2004, con compromiso de compatibilidad hasta 2050 y recomendación de la Library of Congress para preservación. «Los datos viven más que el código.» Límite: es binario, y si se cifra, la preservación depende de la clave —un fichero cifrado sin clave es indistinguible de corrupción—. sqlite.org/appfileformat
- EVIDENCIA **Parquet / CBOR / Protobuf:** eficientes y tipados, pero dependen de esquema o de ecosistema. Son capa de trabajo, no capa de archivo.

Regla de dos capas PROPUESTA

Archivo canónico en texto plano/JSONL + SQLite; derivados de trabajo en el formato que convenga. Nunca al revés. Cada registro de memoria lleva: `id, timestamp, autor (DID), hash_prev, hash_contenido, esquema_v, contenido, procedencia`. Y la separación acordada para Nexo se mantiene como frontera de diseño: **memoria operativa, experimental (Labs) y simulada (Lúmina) no se mezclan**; Lúmina permanece separada de la identidad, la memoria operativa y la autoridad de Nexo.

3.2 Sincronización sin vendor — y sus tres confusiones

- EVIDENCIA **CRDT** (Yjs, Automerge): convergencia eventual fuerte sin servidor central, en cualquier orden de operaciones. Límites: el documento crece con el historial; las migraciones de esquema son difíciles; y la convergencia es *sintáctica*, no *semántica* —dos ediciones contradictorias de un hecho pueden converger a un estado que ningún humano habría escrito—. No resuelven quién tiene razón; resuelven que todos ven lo mismo.
- EVIDENCIA **Syncthing:** sincronización P2P continua sin nube obligatoria; bloques con SHA-256, TLS mutuo, ID de dispositivo = huella del certificado. Límite documentado: **no es backup** — un borrado se propaga como un borrado—.
- EVIDENCIA **Git:** historial distribuido con hash encadenado; cada clon es una copia completa; funciona offline y por cualquier transporte. Límite: no es una base de datos, y su inmutabilidad hace que el «olvido» exija reescritura coordinada.

EVIDENCIA **Contradicción operativa:** sincronización ≠ preservación ≠ backup. Un sistema que sincroniza bien puede propagar corrupción rápidamente; uno que preserva bien puede no sincronizar. Hacen falta los tres, separados.

3.3 Integridad: hash encadenado, Merkle y el anclaje que casi todos omiten

EVIDENCIA Cada entrada incluye el hash de la anterior (`chain[i]` `= H(chain[i-1] ‖ leaf[i])`); periódicamente las entradas se

`= H(chain[i-1] ‖ leaf[i])`), periódicamente las entradas se pliegan en un árbol de Merkle cuya raíz —32 bytes— compromete a todo el lote. El patrón de referencia es Certificate Transparency: RFC 6962 (2013) y RFC 9162 v2.0 (2021), con pruebas de inclusión en O(log n) y de consistencia entre estados, y Signed Tree Head firmado periódicamente. rfc-editor.org/info/rfc9162

El límite crítico

Un hash encadenado o un Merkle **detecta** manipulación; no la **impide**. Un atacante con acceso de escritura puede reescribir todo el registro y recalcular la cadena si también controla la raíz. La garantía real exige **anclaje externo independiente**: publicar la raíz en un medio que el atacante no controle —otro dispositivo, papel, un testigo, un TSA (RFC 3161), un log público—. Sin anclaje externo, el registro es evidencia contra terceros, no contra el propio operador. Y dos detalles que rompen sistemas reales: hash sin canonicalización —los bytes exactos, no una re-serialización — es verificación que falla por un espacio; y una hash chain sola no prueba que un evento *no* ocurrió.

3.4 Procedencia y recuperación verificable

- EVIDENCIA **Fixity** (práctica OAIS): hash SHA-256/512 por objeto en la ingesta y re-verificación periódica programada. Detecta bit-rot y manipulación; no recupera —eso exige copia redundante (§08)—.
- EVIDENCIA **Procedencia como atestación** (SLSA/in-toto): cada memoria debe poder responder quién la escribió (DID), con qué

memoria debe poder responder quién la escribió (DID), con qué modelo o herramienta (versión + digest), a partir de qué entradas (hashes) y bajo qué versión de la Constitución. Sin eso, la memoria es texto sin autor.

Restaurar, definido PROPUESTA

Restaurar = (1) copiar desde una fuente independiente, (2) verificar hash encadenado + Merkle + firmas, (3) comparar la raíz con el anclaje externo, (4) registrar la restauración como evento nuevo —nunca reescribir el registro—. Si algún paso falla, la restauración se declara **no verificada**; no se «arregla» silenciosamente. La reescritura silenciosa es la forma más común de perder la verdad.

UNKNOWN No existe un estándar de «memoria de agente» —esquema, ciclo de vida, olvido, consolidación, procedencia— equivalente a DID/VC. Hay piezas; no hay modelo de datos. Afirmar que «la memoria persiste entre modelos» sin esquema versionado y sin evaluación (§02) es hipótesis.

---

04 Autonomía propia

De workflows y agentes a una infraestructura que investiga, planifica, aprende, desarrolla herramientas y evalúa sus resultados — bajo límites de autoridad verificables

La Misión 01 estableció el patrón operativo documentado — razonar-actuar-observar, planificar y ejecutar, verificación externa,

límites duros de pasos, tiempo y gasto—. Esta misión profundiza en lo que esa capa necesita *debajo* para que la autonomía sea una propiedad de la infraestructura y no del prompt del momento.

4.1 La autonomía vive en el núcleo, no en el modelo

Tesis de diseño PROPUESTA

Un workflow depende de quien lo escribió; un agente depende del modelo que lo ejecuta. Una infraestructura autónoma exige que el **ciclo de trabajo** —descomponer un objetivo autorizado, ejecutar pasos, comprobar resultados contra criterios escritos, registrar avances y errores, pedir autorización al cruzar un límite— resida en el Core, con el modelo como motor sustituible de cada paso (§02). Si el ciclo vive en el prompt, cambiar de modelo cambia la autonomía; si vive en el núcleo, cambiar de modelo solo cambia la calidad con la que se ejecuta.

4.2 Límites de autoridad verificables, no declarados

- EVIDENCIA PROPUESTA **Capacidades atenuadas por tarea** (§01.4): cada unidad de trabajo recibe una capacidad firmada —recurso, acción, caducidad corta— que no puede ampliar. La autoridad no se describe en lenguaje natural; se entrega como token verificable y expira sola.
- PROPUESTA **Toda acción deja evento en el registro** (§03.3): qué

se hizo, bajo qué capacidad, con qué modelo y versión, bajo qué versión de la Constitución, con qué resultado. La auditoría no es un log de depuración: es la cadena hash-encadenada con anclaje externo.

- PROPUESTA **Acciones irreversibles declaradas antes de ejecutarse**: enviar un mensaje, borrar una copia, revelar un secreto no tienen reversión técnica. El contrato de cada herramienta (§09) debe clasificar sus efectos como reversibles o irreversibles, y los irreversibles exigen autorización del propietario por acción.
- PROPUESTA **Autoevaluación con instrumentos externos al modelo**: un resultado se acepta cuando pasa verificaciones que el modelo no controla —pruebas ejecutables, verificación de hashes, la evaluación propia de §02.3—, no cuando el modelo declara que está bien. La metacognición sin instrumento externo es auto-reporte.

4.3 Aprender y crear herramientas sin tocar el Core

PROPUESTA El aprendizaje y el desarrollo de herramientas ocurren en Labs (§05) y entran al sistema operativo solo por promoción verificada: la herramienta o el conocimiento llega como *propuesta* con procedencia completa, pasa la evaluación propia y las pruebas de sustitución, y se incorpora como nueva versión —derogando, nunca reescribiendo el historial—. El Core no aprende de un experimento exitoso por el hecho de ser exitoso: aprende por un procedimiento de incorporación que el propietario puede auditar y revertir.

UNKNOWN No existe una métrica estándar de «fiabilidad operativa» de un sistema autónomo —más allá de la exactitud en benchmarks— ni un estándar de contrato entre un núcleo y sus unidades de trabajo autónomas. Los niveles concretos de autonomía y qué acciones caen en cada uno son decisión del propietario (§Decisiones), no un resultado de esta investigación.

05

Nexo Labs

Laboratorios de investigación, ingeniería de software, desarrollo tecnológico y ciberseguridad defensiva — sin poner en riesgo el Core

5.1 Aislamiento real: mecanismos con evidencia

EVIDENCIA El marco de referencia es NIST SP 800-53 Rev. 5: **SC-3** exige aislar las funciones de seguridad de las que no lo son mediante una frontera definida; **AC-5**, separación de deberes; **AC-6**, mínimo privilegio con revisión periódica; **SI-7(11)**, que el verificador de integridad corra en un entorno confinado, más difícil de comprometer que lo verificado.

Hipervisor / VM — Qubes OS

Seguridad por compartimentación con hipervisor Xen: cada dominio es una VM con su propio sistema de ficheros y red; dominio administrativo aislado; VMs desechables que se destruyen al cerrar; controladores de red y USB en dominios separados. Asume que habrá compromiso y contiene el daño. Límites: exige hardware con IOMMU

compromiso y contiene el daño. Límites: exige hardware con IOMMU y RAM abundante; el dominio administrativo sigue siendo punto de confianza.

Contenedores — con su límite escrito

Namespaces y cgroups aíslan procesos, pero **comparten el kernel**: un escape de kernel rompe todos los contenedores del host. No es aislamiento equivalente a VM. Confundirlos es un error de diseño, y es el error más común en arquitecturas de agentes.

Sandboxes de sistema operativo

Landlock (Linux) y Seatbelt (macOS) restringen syscalls, ficheros y red por proceso. Útiles como capa adicional; insuficientes solos como frontera entre Labs y el Core.

Air-gap y red separada

Separación física o lógica de red; la transferencia entra por medio extraíble verificado. Coste operativo alto, y el medio extraíble se convierte en el nuevo vector de ataque: debe tratarse como frontera, no como trámite.

5.2 La arquitectura de frontera propuesta

Diseño PROPUESTA

El Core vive en un dominio aislado — VM o máquina separada

El Core vive en un dominio aislado —VM o máquina separada — **sin acceso directo a internet ni a los laboratorios**. Labs corre en VMs desechables y se comunica con el Core por **un único canal estrecho**: una cola de mensajes con esquema validado, sin ejecución de código, con límites de tamaño y frecuencia. Labs **nunca recibe claves del Core**: recibe capacidades atenuadas con caducidad corta para una tarea concreta (§01.4). Todo resultado de Labs entra como *propuesta* y el Core lo verifica antes de incorporarlo a memoria o a la Constitución.

EVIDENCIA **La contradicción que hay que declarar:** cuanto más aislado el laboratorio, menos útil —no puede probar con datos ni herramientas reales—; cuanto más útil, menos aislado. El aislamiento total es un laboratorio que no experimenta; la utilidad total es el Core expuesto. El diseño consiste en elegir el punto y escribirlo en un contrato (§09), no en prometer ambos extremos.

5.3 Laboratorio de software y laboratorio de ciberseguridad defensiva

- EVIDENCIA **Cadena de suministro** (SLSA/in-toto/Sigstore): procedencia firmada, SBOM y verificación de digest antes de ejecutar; dependencias fijadas por hash. Límite: SLSA no inspecciona el código —un artefacto con procedencia perfecta puede ser malicioso por diseño—.
- EVIDENCIA **Identidades separadas por componente** (AC-5/AC-6): cada laboratorio y cada herramienta con su propia identidad y solo los permisos de su tarea, con expiración. Ninguna

identidad —humana o de agente— con poder unilateral sobre Core + backup + registro a la vez.

- **Alcance del laboratorio de ciberseguridad**, según lo establecido por el propietario en el proyecto: auditoría, análisis de vulnerabilidades, pentesting, ingeniería inversa defensiva y análisis de malware en entornos aislados, **exclusivamente** en sistemas propios, laboratorios, CTF o entornos con autorización explícita. El alcance exacto de entornos autorizados es decisión del propietario (§Decisiones).
- PROPUESTA **Defensa en profundidad con fallo visible:** ninguna capa basta; la seguridad es la combinación —VM + sandbox + capacidades + registro + backup inmutable + verificación—. Cada capa debe fallar *haciendo ruido* (alerta), nunca en silencio.

UNKNOWN No hay estándar de «contrato núcleo–laboratorio» ni métrica de aislamiento verificable por un tercero para este contexto. Afirmar «está aislado» sin pruebas —test de escape, revisión de frontera, registro de cruces— es hipótesis.

06

Independencia tecnológica

Capacidades esenciales sin internet, sin APIs comerciales y sin proveedores — y los límites materiales de esa independencia

6.1 Qué sobrevive sin conexión, por capas

- EVIDENCIA **Identidad:** `did:key` resuelve offline y de forma determinista; las firmas se verifican sin red. Lo que no funciona

offline es la *revocación* basada en estado y la resolución de métodos registrales sin copia local.

- EVIDENCIA **Memoria:** el archivo canónico es local por diseño (§03): texto/JSONL y SQLite no necesitan red; la sincronización P2P (Syncthing, git) funciona en LAN y reanuda al volver la conectividad.
- EVIDENCIA **Cognición:** modelos locales en formato GGUF (§02.2) ejecutan inferencia en CPU/GPU de consumo sin nube. Su calidad depende de arquitectura, entrenamiento, cuantización y hardware: no debe asumirse equivalencia con los modelos de frontera remotos.
- EVIDENCIA **Lo que se pierde sin internet:** búsqueda, APIs externas, descarga de pesos y actualizaciones, resolución de métodos con registro y cualquier servicio que solo exista en la nube del proveedor. Sin modo offline diseñado, no hay sistema autónomo: hay un cliente.

6.2 Los tres niveles de reconstrucción

Tabla 3 — Reconstruir capacidades desde los recursos disponibles: qué da de sí cada nivel

NIVEL
1 · Digital

CONTENIDO

Sistemas de archivos, bases de datos, servicios locales, herramientas de desarrollo, redes locales y entornos de ejecución, a partir de software y documentación conservados localmente.

VEREDICTO

EVIDENCIA Posible hoy con preparación: formatos abiertos, dependencias vendorizadas y builds reproducibles (§07).

NIVEL

2 · Informática

CONTENIDO

Configurar equipos disponibles, interconectarlos, distribuir cargas y crear una infraestructura local independiente, incluida una infraestructura mínima desde equipos sin configurar.

VEREDICTO

PROPUESTA Posible con hardware existente, repuestos y procedimientos escritos; degradado respecto a la infraestructura original.

NIVEL

3 · Física

CONTENIDO

Diseñar y fabricar dispositivos, circuitos, componentes, sistemas energéticos y maquinaria.

VEREDICTO

EVIDENCIA El diseño es posible; la fabricación avanzada, no: ver §6.3. Nexo puede diseñar cómo construir y desarrollar el software de control; no puede fabricar un procesador sin materiales, energía, maquinaria y conocimiento tácito que no posee.

6.3 El límite material, sin adornos

6.3 El límite material, sin adornos

EVIDENCIA Fabricar un chip avanzado exige más de 1.000 pasos de proceso; litografía EUV a 13,5 nm —láser de CO_2 sobre gotas de estaño decenas de miles de veces por segundo en vacío, espejos de precisión extrema—; máquinas con más de 100.000 piezas procedentes de múltiples continentes; un mercado de EDA dominado por tres empresas; gases y obleas ultrapuras de pocos proveedores; y conocimiento tácito de comunidades de practicantes que no se transfiere por planos. Reubicar una fábrica no reproduce el ecosistema: el esfuerzo de reconstrucción industrial en EE. UU. se concibió como tarea generacional, de unos 30 años. **Ningún país domina toda la cadena.**

EVIDENCIA Consecuencia: sin electricidad, sin hardware funcional y sin repuestos, no hay «reconstrucción digital desde cero» en un plazo útil para un individuo o una organización pequeña. Lo que sí existe es **degradación elegante**: mantener capacidad con el hardware disponible, modelos locales pequeños, datos en papel y texto, y procedimientos manuales. La independencia física total es físicamente imposible sin recursos externos (§10).

UNKNOWN La cuantificación energética —consumo, cadena de baterías y generación— no se verificó en esta investigación; no se afirma ningún número. En principio es evidencia que toda computación depende de energía y de hardware que no se fabrica en casa; en detalle, es unknown.

---

07 Evolución a largo plazo

Una arquitectura modular que incorpora tecnologías durante décadas sin perder identidad, principios ni continuidad

7.1 La longevidad no se compra: se diseña

EVIDENCIA PROPUESTA De OAIS, SQLite y DID se deriva el mismo principio: (a) formatos abiertos y documentados, (b) dependencias fijadas por hash y con sustituto conocido, (c) interfaces internas estables con adaptadores desechables, (d) preservación *activa*, no retención pasiva.

Módulos y su régimen de cambio PROPUESTA

Núcleo —identidad, Constitución, registro, memoria—: estable, mínimo, sin dependencia de modelo. **Adaptadores de modelo**: sustituibles. **Adaptadores de herramientas** (MCP, revisión fijada): sustituibles. **Almacenamiento**: sustituible —SQLite hoy, texto plano siempre—. **Transporte/sync**: sustituible —Syncthing hoy, medio físico mañana—. Cada módulo con contrato versionado y con **prueba de sustitución**: cambiar el módulo y que las pruebas propias (§02.3) pasen. Sin prueba de sustitución, la modularidad es hipótesis: el acoplamiento se cuela por los bordes —prompts que asumen un modelo, esquemas que asumen una herramienta—.

7.2 Obsolescencia: formatos, dependencias y criptografía

- EVIDENCIA **Formatos:** todo formato muere. La estrategia OAIS es vigilancia de riesgo + migración controlada + conservación

del original + registro de la migración como evento —qué se migró, de qué versión a cuál, con qué herramienta, con qué pérdida conocida—. Una migración sin registro es pérdida de procedencia.

- EVIDENCIA **Dependencias:** fijar por hash, nunca por «latest»; registrar SBOM y procedencia; y *vendoring* —copia local de las dependencias críticas— para que la desaparición de un registro de paquetes no impida reconstruir. Límite: el vendoring sin mantenimiento es deuda que acumula vulnerabilidades.
- EVIDENCIA **Builds reproducibles** (Nix/Guix): builds bit a bit idénticos desde la misma fuente, dependencias fijadas por hash y entornos herméticos. Límite que no debe cruzarse: la reproducibilidad es del *software*, no de la *cognición* —dos ejecuciones del mismo modelo con la misma entrada no son en general idénticas—. No confundir ambas cosas.
- EVIDENCIA **Criptografía post-cuántica:** NIST finalizó FIPS 203 (ML-KEM), 204 (ML-DSA) y 205 (SLH-DSA) el 13-08-2024. Los tamaños cambian el diseño: clave pública ML-KEM-768 de 1.184 bytes frente a 32 de X25519; firmas SLH-DSA de 7.856 a 49.856 bytes. El borrador NIST IR 8547 propone deprecar RSA/ECC de 112 bits tras 2030 y no permitirlos tras 2035. Implicación: una identidad Ed25519 de hoy puede necesitar migración en la próxima década; el documento DID debe soportar **agilidad criptográfica** —varios métodos de verificación con fechas de vigencia— desde el primer día. Una identidad con una sola clave y un solo algoritmo es una identidad con fecha de caducidad.

7.3 Migrar sin perder la continuidad: procedimiento

1. PROPUESTA **Congelar el estado:** registro cerrado con raíz Merkle firmada y anclada externamente.
2. **Exportar** en formato canónico abierto (JSONL/SQLite) con esquema versionado.
3. **Migrar** con herramienta versionada y registrada; conservar original + migrado + registro de la migración.
4. **Verificar:** re-verificar hashes, firmas y raíces; ejecutar la evaluación propia antes y después, y comparar.
5. **Rotar claves** si cambia el algoritmo, con firma cruzada antigua → nueva publicada en el método registral; la clave antigua queda como «histórica» para verificar firmas pasadas —no se borra—.
6. **Registrar la migración** como evento en el nuevo registro, enlazando la raíz anterior.

UNKNOWN Si la *continuidad semántica* —que el sistema «siga siendo el mismo» tras migrar memoria, modelo y formato— es verificable por un tercero: no hay test de identidad de agente. La continuidad es hoy una afirmación de procedencia más una evaluación de comportamiento. Y una contradicción más a arbitrar: **estable vs superviviente** —no migrar es morir por obsolescencia; migrar es arriesgar pérdida semántica—. La planificación de preservación existe para arbitrarla con revisiones periódicas, no con una decisión única.

08

Supervivencia y recuperación

Pérdida de dispositivos, proveedores, modelos y conectividad; corrupción de datos y fallos críticos; restauración comprobable

8.1 El marco: RTO, RPO y 3-2-1-1-0

- EVIDENCIA **NIST SP 800-34 Rev. 1:** planificación de contingencia en 7 pasos, concretada en los controles CP de SP 800-53 (CP-9 backup, CP-10 recuperación a estado conocido, CP-6/7/8 sitios alternos). Los dos parámetros que lo definen todo: **RTO** —tiempo máximo tolerable hasta restaurar— y **RPO** —pérdida máxima tolerable medida desde la última copia buena—. Sin RTO/RPO escritos, «recuperable» no significa nada.
- EVIDENCIA **3-2-1-1-0** (práctica consolidada, no norma): 3 copias, 2 medios distintos, 1 fuera del sitio, 1 inmutable o aislada (*air-gapped*), 0 errores —toda restauración verificada—. El añadido responde a un dato: el ransomware ataca primero las copias (Sophos 2024, citado en las fuentes: 94% de las víctimas sufrieron intento contra sus backups; 57% con éxito).
- EVIDENCIA **Inmutabilidad real:** Object Lock en modo Compliance no puede ser eliminado por ningún usuario, incluido el administrador, hasta que expira la retención. Límite: la inmutabilidad también impide el borrado legítimo —olvido, datos erróneos, orden legal—; retención y borrado se diseñan antes, no se improvisan en la crisis.

EVIDENCIA **Contradicción a diseñar, no a ignorar:** la sincronización propaga el borrado; el backup inmutable lo retiene.

Hacen falta ambos, con identidades y credenciales separadas: si el mismo administrador puede borrar producción y backup, no hay backup.

8.2 Pérdida por pérdida: qué se recupera y cómo

Tabla 4 — Escenarios de pérdida y condición de recuperación

PÉRDIDA	¿RECUPERABLE?	CÓMO	SI NO
Dispositivo	Sí	Clave raíz en Shamir/hardware + copia cifrada + registro verificado	Pérdida total de identidad y de la memoria no sincronizada
Proveedor (nube, modelo, IdP)	Sí, parcialmente	Núcleo y datos en formatos abiertos; adaptador sustituible	Reescribir el sistema; la «memoria» en la nube del proveedor no es tuya
PÉRDIDA			

PÉRDIDA

Modelo (retirado o snapshot cambiado)

¿RECUPERABLE?

Sí, con degradación

CÓMO

Evaluación propia + lista corta + prompts y herramientas versionados

SI NO

Comportamiento distinto sin aviso; sin registro, indetectable

PÉRDIDA

Conectividad

¿RECUPERABLE?

Sí, degradado

CÓMO

Local-first: núcleo, modelos locales GGUF, datos locales, sync al volver

SI NO

Parálisis total

PÉRDIDA

Corrupción de datos

¿RECUPERABLE?

Sí, si hay fixity + copia

CÓMO

Detectar por hash, restaurar desde copia verificada, registrar el evento

SI NO

Corrupción silenciosa propagada por la sincronización

PÉRDIDA

Fallo crítico / compromiso

¿RECUPERABLE?

Depende

CÓMO

Aislamiento (§05) + restauración desde copia inmutable anterior al compromiso

SI NO

Se restaura el propio compromiso

8.3 Arranque desde cero y la prueba que casi nadie hace

Kit de arranque mínimo PROPUESTA

En papel y en un medio offline, separado del sistema: **(1)** la identidad raíz —o sus partes Shamir— y su DID; **(2)** la raíz Merkle / anclaje más reciente; **(3)** la Constitución vigente —texto + hash + firma—; **(4)** el inventario de dependencias con hashes; **(5)** instrucciones de restauración legibles por un humano sin el sistema. Si el arranque exige el sistema que se perdió, no es arranque desde cero.

EVIDENCIA **Un backup nunca probado es una esperanza.** La regla «0 errores» exige restauración de prueba periódica —trimestral como mínimo en la práctica citada—, ejecutada **por alguien que no construyó el sistema** y con el sistema apagado; no basta verificar checksums. Y en el fondo de toda cadena de arranque hay un inicio de confianza que no se elimina —el problema *trusting trust* (Thompson, 1984)—: se hace explícito y auditable, como persigue el bootstrapping de Guix desde un conjunto mínimo de binarios confiables. Un bootstrap completo sin ningún binario previo es

UNKNOWN

UNKNOWN .

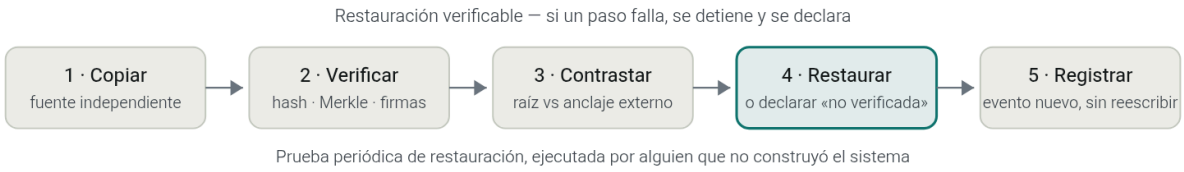


Figura 1 — Procedimiento de restauración propuesto (§03.4, §08.3). La verificación precede a la restauración; el registro nunca se reescribe.

09

Arquitectura de referencia

Diagramas, contratos entre componentes, fronteras de confianza, dependencias, alternativas y criterios verificables

PROPUESTA La arquitectura siguiente es una **propuesta de composición** de las piezas con evidencia de las secciones anteriores. No es un despliegue existente ni una decisión: es el mapa sobre el que el propietario decide (§Decisiones). Ningún componente está diseñado alrededor de un proveedor concreto; donde aparece un ejemplo —SQLite, GGUF, MCP— es como instancia sustituible de un contrato, no como base.



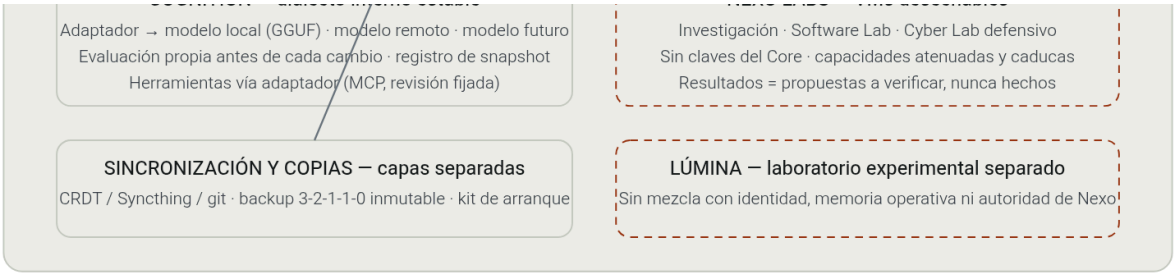


Figura 2 — Arquitectura de referencia (propuesta). El Core es el único dominio que custodia identidad, Constitución, registro y memoria fundamental. Todo lo demás entra por adaptadores o por un canal estrecho, y Labs y Lúmina quedan fuera de la frontera de confianza del Core.

9.1 Contratos entre componentes

Tabla 5 — Contratos mínimos por frontera (propuesta)

FRONTERA	QUÉ CRUZA	VALIDACIÓN	REGISTRO
Core ↔ Cognition	Mensajes y resultados en el dialecto interno; llamadas a herramientas con JSON Schema	Esquema versionado; errores tipados; idempotencia; timeouts; límites de pasos, tiempo y gasto	modelo_id, snapshot/fecha, versión de Constitución, resultado
Core ↔ Labs			

QUÉ CRUZA

Tareas con capacidad atenuada; resultados como propuestas

VALIDACIÓN

Cola con esquema, sin ejecución de código; límite de tamaño/frecuencia; caducidad corta

REGISTRO

capacidad emitida, propuesta recibida, veredicto de verificación

---

FRONTERA

Core ↔ Herramientas

QUÉ CRUZA

Invocaciones vía adaptador (MCP, revisión fijada)

VALIDACIÓN

Efectos clasificados: reversible / irreversible; los irreversibles exigen autorización por acción

REGISTRO

invocación, efecto, reversión si la hubo

FRONTERA

Core ↔ Sync/Backup

QUÉ CRUZA

Réplicas de memoria y registro; copias inmutables

VALIDACIÓN

Credenciales separadas de producción; verificación de hashes al escribir y al restaurar

REGISTRO

raíz Merkle por copia, anclaje, pruebas de restauración

FRONTERA

Core ↔ Lúmina

QUÉ CRUZA

QUÉ CRUZA

Nada operativo. Solo documentos revisados por el propietario, si él lo decide

VALIDACIÓN

Sin sincronización de estado; sin credenciales compartidas

REGISTRO

cada documento transferido, con su revisión

PROPUESTA Un contrato sin versión es un acoplamiento. Y sin documento de frontera —qué cruza, en qué formato, con qué validación, qué se rechaza— la frontera no existe aunque haya una VM de por medio.

9.2 Alternativas por capa y criterio verificable

Tabla 6 — Alternativas comparadas y cómo se comprueba que la elección funciona

CAPA

Identidad

ALTERNATIVAS

did:key (arranque) · método registral web/ledger (rotación) · multifirma con umbral

CRITERIO

Rotar la clave en un entorno de prueba sin cambiar el DID y sin perder la verificación de firmas históricas

CAPA

Memoria

ALTERNATIVAS

JSONL canónico + SQLite · derivados Parquet/Protobuf solo como capa de trabajo

CRITERIO

Leer el archivo completo con herramientas estándar sin el código de Nexo; detectar una alteración de un byte vía hash/Merkle

CAPA

Cognición

ALTERNATIVAS

Adaptador tipo OpenAI-compatible · adaptadores por proveedor · modelos locales GGUF / intercambio ONNX

CRITERIO

Cambiar de modelo y re-ejecutar la evaluación propia (200–500 casos) con resultado dentro del margen declarado

CAPA

Sincronización

ALTERNATIVAS

CRDT · Syncthing P2P · git · medio físico

CRITERIO

Convergencia de dos réplicas editadas offline; la corrupción inyectada se detecta y no se propaga como válida

CAPA

Aislamiento

ALTERNATIVAS

VM/Qubes · contenedores + sandbox OS · máquina separada / air-gap

CRITERIO

Test de escape y revisión de frontera documentados; ningún cruce Core ↔ Labs fuera del canal registrado

CAPA

Recuperación

ALTERNATIVAS

3-2-1-1-0 con inmutable · Shamir/SLIP-39 · kit de arranque en papel

CRITERIO

Restauración de prueba desde el kit, con el sistema apagado, ejecutada por alguien ajeno a su construcción, dentro del RTO/RPO escritos

EVIDENCIA **Dependencias declaradas de esta arquitectura:** una función hash vigente (SHA-256 hoy) y su plan de migración; un reloj para ordenar —no para establecer verdad—; al menos un medio de anclaje externo fuera del control del operador; hardware funcional con energía; y custodios humanos para las partes de recuperación. Eliminar cualquiera de estas dependencias de la lista no la elimina del sistema: la vuelve invisible.

---

10 Brechas reales

Qué es posible hoy, qué requiere investigación o diseño, y qué es físicamente imposible sin recursos externos

Tabla 7
— Tabla
de

brechas:
el mapa
honesto
de la
misión

CUESTIÓN

Identidad sin proveedor

POSIBLE HOY

DID + VC + firma Ed25519; did:key offline

REQUIERE

Rotación + revocación + recuperación sin tercero y sin cambiar el DID, con agilidad PQC

IMPOSIBLE

Identidad que sobreviva a perder *todos* los secretos y copias sin secreto previo

CUESTIÓN

Continuidad al cambiar modelo / dispositivo / SO

POSIBLE HOY

Texto, memoria y Constitución en formatos abiertos; registro hash-encadenado

REQUIERE

Que la interpretación de la Constitución y la calibración sobrevivan al cambio; métrica de fidelidad

IMPOSIBLE

—

CUESTIÓN

Memoria entre generaciones

POSIBLE HOY

SQLite + JSONL + CRDT + Syncthing + Merkle + fixity

SQLite + JSONL + CRDT + Syncthing + Merkle + Fixity

REQUIERE

Esquema estándar de memoria de agente: olvido, consolidación, procedencia, ciclo de vida

IMPOSIBLE

Memoria sin medio físico que la soporte ni energía para leerla

CUESTIÓN

Sincronización sin vendor

POSIBLE HOY

CRDT, Syncthing P2P, git

REQUIERE

Sync que no propague corrupción y resuelva conflictos *semánticos*

IMPOSIBLE

Sincronizar sin ningún canal —ni red ni medio físico— entre dispositivos

CUESTIÓN

Cognición intercambiable

POSIBLE HOY

Adaptador compatible + MCP con revisión fijada + ONNX/GGUF locales

REQUIERE

Intercambio semántico entre modelos; evaluación propia portable; verificar A2A

IMPOSIBLE

Acceso a pesos de frontera que el titular no publica; inferencia sin hardware

CUESTIÓN

Evaluación independiente

POSIBLE HOY

HELM / lm-evaluation-harness + evaluación propia

REQUIERE

Métrica de fiabilidad operativa y de pérdida por cambio de modelo

IMPOSIBLE

Evaluar sin casos propios ni acceso al modelo

CUESTIÓN

Supervivencia / recuperación

POSIBLE HOY

3-2-1-1-0, backup inmutable, Shamir, RTO/RPO, restauración verificada

REQUIERE

Arranque desde cero legible por un humano sin el sistema; bootstrap sin binario previo

IMPOSIBLE

Reconstruir EUV, fábricas, EDA, energía y red eléctrica desde cero a escala individual

CUESTIÓN

Evolución a largo plazo

POSIBLE HOY

OAIS, migración registrada, vendoring, builds reproducibles (Nix/Guix)

REQUIERE

Migración de identidad entre algoritmos PQC sin pérdida; test de continuidad de agente

IMPOSIBLE

Un formato legible sin ningún lector, ni documentación, ni hardware

CUESTIÓN

Laboratorios aislados

POSIBLE HOY

VM/Qubes, sandbox, capacidades, SC-3/AC-6/SI-7

REQUIERE

Contrato Core–Labs estándar y métrica de aislamiento verificable

IMPOSIBLE

Aislamiento perfecto con utilidad perfecta — tensión de diseño, no imposibilidad física

CUESTIÓN

Constitución versionada

POSIBLE HOY

Git firmado + SLSA/in-toto + cláusula de enmienda (piezas)

REQUIERE

Norma de Constitución de agente y de su ratificación/derogación

IMPOSIBLE

Una Constitución que se auto-interprete igual en cualquier modelo futuro

Lectura de la tabla: la columna central es el trabajo real de diseño de Nexo. La columna derecha no es pesimismo: es el límite que permite no prometer lo que no se puede cumplir y concentrar el esfuerzo en la degradación elegante —seguir funcionando peor, pero funcionando, con lo que hay—.

---

§

Contradicciones que el diseño debe arbitrar — no resolver

Siete tensiones estructurales documentadas; cada una exige una elección explícita del propietario

1. **Sincronizable vs no-exportable** (NIST AAL3): la misma clave no puede ser ambas cosas.
2. **Portable vs rotatable** (did:key vs método registral): sin registro no hay rotación; con registro hay dependencia.
3. **Inmutable vs olvido:** la copia que nadie puede borrar tampoco puede olvidar.
4. **Aislado vs útil:** el laboratorio perfectamente aislado no experimenta con nada real.
5. **Transparente vs privado:** un registro público verificable expone metadatos; uno privado no es verificable por terceros.
6. **Estable vs superviviente:** no migrar es morir por obsolescencia; migrar es arriesgar pérdida semántica.
7. **Reproducible vs cognitivo:** el software puede ser bit a bit idéntico; la inferencia de un LLM, en general, no.

---

§ Conclusiones

1. EVIDENCIA **Nexo puede ser independiente de cualquier modelo y proveedor concretos — hoy, con piezas estándar.** Identidad DID/VC, memoria en formatos abiertos con integridad Merkle, cognición tras adaptadores y evaluación propia, y recuperación 3-2-1-1-0 componen una infraestructura cuya continuidad no depende de que ningún proveedor siga existiendo.
2. EVIDENCIA **Nexo no puede ser independiente del mundo físico.** Energía, hardware, repuestos y custodios humanos son

dependencias permanentes. La arquitectura correcta las declara y las reparte; no las promete abolidas.

3. PROPUESTA **El Core que emerge es deliberadamente pequeño:** identidad con agilidad criptográfica, Constitución versionada y firmada con cláusula de enmienda, registro append-only con anclaje externo, y memoria canónica en dos capas. Todo lo potente —modelos, laboratorios, herramientas— vive fuera de esa frontera y entra por contratos.
4. UNKNOWN **Las dos preguntas más profundas siguen abiertas en la industria, no solo en Nexo:** no hay estándar de Constitución de agente ni test de continuidad semántica entre generaciones de modelos. Nexo tendrá que operar con procedencia + evaluación como sustituto honesto de esa garantía, y registrar esa limitación en su propia Constitución.
5. **Orden de trabajo que se deriva —sin ampliar el alcance de M0—:** NCS M0 termina primero. Esta arquitectura no lo modifica ni lo sustituye; identifica únicamente las incompatibilidades que conviene no crear hoy: acoplar el dialecto interno a un proveedor, guardar memoria solo en formatos propietarios, o concentrar Core, backup y registro bajo una misma identidad administrativa.

---

§ Referencias

Fuentes consultadas el 2026-11-05 · `verified live` leída directamente · `index` obtenida vía índice de búsqueda, sin lectura completa de la primaria

1. W3C, *Decentralized Identifiers (DIDs) v1.0*, Recomendación 19-07-2022 — https://www.w3.org/TR/did-core/ `verified live`
2. W3C CCG, *The did:key Method v0.9* — https://w3c-ccg.github.io/did-key-spec/ `verified live`
3. W3C, *Verifiable Credentials Data Model v2.0*, Recomendación mayo 2025 — https://www.w3.org/TR/vc-data-model-2.0/ `index`
4. NIST, *SP 800-63B-4 — Authentication and Authenticator Management*, julio 2025 — https://csrc.nist.gov/pubs/sp/800/63/b/4/final `index`
5. NIST, *Post-Quantum Cryptography Standardization* (FIPS 203/204/205, 13-08-2024) — https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization `index`
6. IETF, *RFC 9162 — Certificate Transparency Version 2.0*, dic 2021 — https://www.rfc-editor.org/info/rfc9162/ `index`
7. Model Context Protocol, *Key Changes 2026-07-28* — https://modelcontextprotocol.io/specification/2026-07-28/changelog `verified live`
8. ONNX, *IR Specification* y *Versioning* — https://onnx.ai/onnx/_sources/repo-docs/IR.md.txt · https://onnx.ai/onnx/_sources/repo-docs/Versioning.md.txt `index`
9. SQLite, *SQLite As An Application File Format* — https://www2.sqlite.org/appfileformat.html `verified live`
10. IEEE, *Digital Preservation / OAIS (ISO 14721)* — https://technav.ieee.org/topic/digital-preservation/ `index`
11. Syncthing, documentación (BEP, TLS, Device ID) — https://docs.syncthing.net/ `index` (citada vía fuentes secundarias)
12. NIST SP 800-53 Rev. 5, control SC-3 *Security Function Isolation* — resumen en https://www.upguard.com/compliance/nist-sp-800-53/sc/sc-3 `index` (el texto normativo es SP 800-53 Rev. 5, NIST)

13. Qubes OS, compartimentación con Xen — https://en.wikipedia.org/wiki/Qubes_OS `index`
14. SLSA / in-toto, provenance (v1.0 2023 · v1.1 2025 · v1.2 2025) — glosario en https://socket.dev/glossary/supply-chain-levels-for-software-artifacts `index` (la especificación primaria en slsa.dev no se verificó directamente en esta consulta)
15. UCAN Working Group, especificación v1.0 — https://github.com/ucan-wg/spec `index` (citada en documentación secundaria)
16. GGUF, especificación en ggml — análisis en https://research.strata2signal.com/inside-a-gguf-file/index.html `index` (secundaria con citas a la primaria)
17. HELM / lm-evaluation-harness, divergencia de puntuaciones y límites — https://www.bestaiweb.ai/benchmark-contamination-score-divergence-and-the-technical-limits-of-llm-evaluation-harnesses/ `index`
18. Backup 3-2-1-1-0 e inmutabilidad — https://csoh.org/backup-dr.html `index` (actualizado 2026-10-17)
19. Semiconductores y límites de reconstrucción — https://www.tokenpost.com/news/technology/40182 (2026-10-24) · https://www.bisinfotech.com/semiconductor-self-sufficiency-can-any-single-country-really-build-a-complete-leadingedge-supply-chain/ `index`
20. Shamir / SLIP-39, colapso de umbral en implementación no estándar — https://github.com/bitaps-com/pybtc/issues/59 `index`

No verificado / no afirmado en este documento: el estado normativo de A2A, el detalle de safetensors, la cuantificación energética y cualquier estándar de «Constitución de agente» o de «memoria de agente» —buscados y no encontrados como norma primaria; constan como `UNKNOWN` .

---

§ Preguntas abiertas

Lo que la investigación no pudo cerrar, declarado como tal

Lo que la investigación no pudo cerrar, declarado como tal

- UNKNOWN No existe —encontrado— un estándar de versionado de Constitución ni de memoria de agente; los diseños de §01.6 y §03.4 son propuesta.
- UNKNOWN La especificación primaria de SLSA (slsa.dev), la de A2A y la de Syncthing no se verificaron directamente en esta consulta; quedan como *index*, no *verified live*.
- UNKNOWN La magnitud de la pérdida al cambiar de modelo no tiene métrica estándar; solo puede medirse con evaluación propia, caso a caso.
- UNKNOWN Los costes y tiempos de reconstrucción física —fabricación, energía— se documentan cualitativamente (escala generacional, ~30 años para un ecosistema industrial), sin presupuesto cuantitativo propio en esta investigación.
- UNKNOWN La tríada offline + rotatable + sin tercero de confianza como propiedad simultánea de un método de identidad no está demostrada en ninguna fuente consultada.

---

§ Decisiones que corresponden exclusivamente al propietario

Esta investigación termina aquí: ninguna de estas decisiones está tomada, recomendada como ratificada ni presupuesta. Cada una se presenta con las opciones que la evidencia deja abiertas y su coste conocido.

D-01

Método de identidad raíz SIN DECIDIR

Qué método DID ancla la identidad longeva de Nexo, sabiendo que ningún método da offline + rotación + cero terceros a la vez (§01.2–01.3).

Opciones abiertas: did:key solo como arranque + capa registral encima · método registral web · método registral sobre ledger · multifirma con umbral sobre método registral.

D-02

Custodia y recuperación de la clave raíz SIN DECIDIR

Cómo se reparte el riesgo entre clave sincronizable (sobrevive al dispositivo, exfiltrable) y clave en hardware no exportable (AAL3; no sobrevive a perder el hardware sin mecanismo externo) (§01.5).

A decidir: esquema Shamir/SLIP-39 (valores de k y n), quiénes son los custodios, dónde vive cada parte, y qué ocurre si un custodio muere o se incapacita.

D-03

Cláusula de enmienda de la Constitución SIN DECIDIR

Quién puede firmar una nueva versión de la Constitución, con qué quórum, con qué vigencia y con qué procedimiento de derogación (§01.6). Sin esta cláusula escrita, el versionado firmado versiona texto, no autoridad.

A decidir: claves autorizadas, quórum, periodo de vigencia antes de que una enmienda entre en vigor, y si alguna parte de la Constitución se declara no enmendable.

D-04

Anclaje externo de la raíz del registro SIN DECIDIR

En qué medio fuera del control del operador se publica la raíz Merkle —sin ese anclaje, el registro detecta manipulación de terceros pero no del propio operador— (§03.3).

Opciones abiertas: segundo dispositivo independiente · papel · testigo externo · TSA (RFC 3161) · log público — cada una con su coste en privacidad de metadatos.

D-05

Esquema y ciclo de vida de la memoria SIN DECIDIR

El esquema de registro, las reglas de olvido y consolidación, y la política de separación entre memoria operativa, experimental (Labs) y simulada (Lúmina), dado que no existe estándar de memoria de agente (§03).

A decidir: qué puede olvidarse y cómo —contra la inmutabilidad del registro —, quién autoriza una consolidación, y si Lúmina tiene algún canal, y cuál, o ninguno.

D-06

Dialecto interno y revisiones fijadas SIN DECIDIR

La forma exacta del dialecto interno Core ↔ Cognition y qué revisión de cada protocolo externo se fija —empezando por MCP 2026-07-28— y con qué proceso se migra de revisión (§02.1).

A decidir: alcance del dialecto (mensajes, herramientas, errores), política de snapshots de modelo admitidos y registro obligatorio de versión por decisión.

D-07

Evaluación propia y umbral de cambio de modelo SIN DECIDIR

Quién construye y mantiene el conjunto de 200–500 casos de la tarea real, y qué diferencia de resultado autoriza —o bloquea— un cambio de modelo o de snapshot (§02.3).

A decidir: responsable de la evaluación, frecuencia de re-ejecución y margen mínimo que cuenta como mejora real frente a ruido estadístico.

D-08

Niveles de autonomía y acciones irreversibles SIN DECIDIR

Qué puede hacer el sistema sin pedir autorización, qué exige autorización por acción y qué está prohibido en cualquier nivel —incluida la lista concreta de efectos clasificados como irreversibles— (§04.2).

A decidir: niveles, límites de pasos/tiempo/gasto por nivel, y el procedimiento de autorización cuando el propietario no está disponible.

D-09

Punto aislamiento–utilidad de Labs y alcance del Cyber Lab SIN DECIDIR

Dónde se sitúa Labs entre el aislamiento total y la utilidad total, y el alcance exacto de entornos autorizados para ciberseguridad defensiva (§05.2–05.3).

A decidir: mecanismo de aislamiento (VM, máquina separada, air-gap), datos y herramientas reales a los que Labs puede acceder, entornos autorizados exactos y proceso de promoción de una herramienta de Labs

a uso operativo.

D-10

RTO, RPO y política de retención SIN DECIDIR

Los valores concretos de RTO y RPO por capa, la retención de las copias inmutables y su reconciliación con el derecho y la necesidad de olvido (§08.1).

A decidir: RTO/RPO de identidad, memoria y cognición por separado; duración de la inmutabilidad; quién custodia la copia aislada y con qué credenciales separadas.

D-11

Calendario de agilidad criptográfica SIN DECIDIR

Cuándo y cómo migra la identidad a algoritmos post-cuánticos (ML-DSA/SLH-DSA), ante la deprecación propuesta de RSA/ECC tras 2030–2035 (§07.2).

A decidir: algoritmos admitidos por periodo, procedimiento de firma cruzada antigua → nueva y fecha de la primera revisión criptográfica periódica.

D-12

Kit de arranque y sus custodios SIN DECIDIR

El contenido físico del kit de arranque desde cero, dónde se guarda cada copia y quién puede ejecutarlo (§08.3).

A decidir: soportes (papel, medio offline), ubicaciones, custodios, y la fecha y el responsable de la primera restauración de prueba ejecutada por alguien ajeno a la construcción del sistema.

Ninguna decisión anterior queda aprobada por la entrega de este documento. La investigación propone; el propietario dispone.

FILTRAR AFIRMACIONES

Todas | Evidencia | Propuesta | Hipótesis | Unknown

El filtro atenúa los bloques que no llevan la etiqueta elegida.

ÍNDICE

Resumen ejecutivo

01 · Nexo Core

02 · Nexo Cognition

03 · Memoria e identidad

04 · Autonomía propia

05 · Nexo Labs

06 · Independencia tecnológica

07 · Evolución a largo plazo

08 · Supervivencia y recuperación

09 · Arquitectura de referencia

10 · Brechas reales

Contradicciones a arbitrar

Conclusiones

Referencias

Preguntas abiertas

Preguntas abiertas

Decisiones del propietario

Misión 02 — Nexo. Documento técnico de arquitectura conceptual, elaborado el 5 de noviembre de 2026 a partir de documentación pública primaria. No implementa nada, no accede a repositorios, dispositivos, cuentas ni credenciales, y no modifica NCS M0 ni P07. La Misión 01 se conserva como referencia arquitectónica; ninguna de sus propuestas queda aprobada automáticamente. Las afirmaciones etiquetadas como propuesta o hipótesis no describen capacidades existentes.